

المحاضرة 4

المتحولات والثوابت والأنظمة العددية في ++C

د. سهيل الحمود

د. أسامه ناصر

2025-2026

الفهرس

- المتحولات Variables
- الثوابت Constants
- مجالات الرؤية Variable Scopes
 - محلي Local
 - عام Global
- أنواع المتحولات
- تمثيل الأعداد بالثنائي
- من الثنائي للعشري
- من العشري للثنائي
- عمليات الجمع والطرح
- Underflow و Overflow

المتحولات Variables

- المتحول هو مكان في الذاكرة يُخزّن قيمة مؤقتة أثناء تنفيذ البرنامج.
- لكل متحول:
 - اسم
 - نوع Type
 - قيمة Value
 - موقع في الذاكرة Address
- مثال:

```
int age = 20;  
float salary = 4500.75;  
char grade = 'A';
```

ما هي الثوابت؟

- الثوابت هي قيم لا يمكن تغييرها أثناء تنفيذ البرنامج.
- تساعد في زيادة الأمان، وضوح الكود، وسهولة الصيانة.
- نستخدمها عندما نحتاج قيمة ثابتة لا يجب أن تتغير.

تعريف الثوابت باستخدام `const`

```
const int MAX_USERS = 100;  
const float PI = 3.14159;  
const char GRADE = 'A';
```

- الكلمة المحجوزة `const` تمنع تغيير القيمة.
- يجب إعطاء الثابت قيمة عند تعريفه مباشرة.
-

تعريف الثوابت باستخدام `define`

```
#define PI 3.14159  
#define MAX_SIZE 256
```

- هذا توجيه لمعالج النصوص (Preprocessor).
- لا يقوم بفحص النوع (No type checking).
- يتم استبداله نصيًا قبل عملية الترجمة.

الفرق بين `const` و `#define`

#define	const	الميزة
✗ لا	✓ نعم	فحص النوع
✗ أصعب	✓ أسهل	سهولة التتبع أثناء التصحيح
✗ لا	✓ نعم	احترام مجال الرؤية
✗ لا (استبدال نصي فقط)	✓ نعم	التخزين في الذاكرة

الثوابت التعبيرية (constexpr)

```
constexpr int SIZE = 10;  
constexpr int VALUE = SIZE * 2;
```

- يتم حسابها وقت الترجمة.
- أسرع وأكثر كفاءة.

الفرق بين

`const` و `constexpr` في ++C

- كلاهما يُستخدم لتعريف ثوابت.
- لكن الفرق الأساسي:
- `const` : قد تُحدد قيمته وقت الترجمة أو وقت التشغيل.
- `constexpr` : يجب أن تُحدد قيمته وقت الترجمة فقط.

مثال عن const

- يعرف قيمة لا يمكن تغييرها.
- قد تعتمد قيمته على حساب يتم وقت التشغيل.

```
int a;  
cin >> a; // قيمة runtime  
const int x = a; // مسموح
```

- لا يضمن أن القيمة ثابتة وقت الترجمة.

مثال عن constexpr

- يتضمن أن القيمة تُحسب وقت الترجمة.
- يجب أن تكون القيمة معروفة مسبقاً.

```
constexpr int x = 10; // صحيح
```

- لا يمكن استخدام قيم runtime معه.

```
int a;  
cin >> a;  
constexpr int x = a; // خطأ
```

الثوابت الحرفية (Literal Constants)

أمثلة:

```
10 // ثابت عددي من نوع int
3.14 // ثابت من نوع double
'A' // ثابت محرفي
true // ثابت منطقي
"Hello" // ثابت نصي
```

ثوابت التعداد (Enum Constants)

```
enum Direction {NORTH,SOUTH,EAST,WEST};
```

```
Direction d;  
d = NORTH;
```

```
if (d == NORTH) {  
    cout << "Going North" << endl;  
}
```

- ينشئ مجموعة من القيم الثابتة المرتبطة ببعضها.

لماذا نستخدم الثوابت؟

- منع تغيير القيم المهمة بالخطأ.
- جعل الكود أوضح وأسهل قراءة.
- تسهيل عملية صيانة البرامج.
- تحسين أداء البرنامج عبر التحسينات التي يجريها المترجم.

مجالات الرؤية Variable Scopes

مجال الرؤية هو جزء البرنامج الذي يمكن فيه الوصول للمتحول. هنالك نوعان من المتغيرات وفق مجال الرؤية:

- متغيرات محلية
- متغيرات عامة

المتغيرات المحلية Local Variables

- تُعرّف داخل تابع أو كتلة.
- ينتهي وجودها بانتهاء التابع.

```
void test(){  
    int x = 10; // متغير محلي داخل تابع  
}  
  
{  
    int y = 20; // متغير محلي داخل كتلة  
}  
  
// غير معروفان هنا x , y
```


المتحولات العامة Global

- تُعرّف خارج جميع التوابع.
- يمكن الوصول لها من جميع أجزاء البرنامج.

```
// file1.cpp
#include <iostream>
using namespace std;
int globalVar = 100; // definition

// file2.cpp
#include <iostream>
using namespace std;
extern int globalVar; // declaration
void display() {
    cout << globalVar << endl;
}
```

أنواع المتحولات

- رقمية
- محارف
- عائمة
- بوليانية
- مؤشرات
- مركبة (سنتعرف عليها لاحقاً)

النظام الثنائي Binary System

- الحاسوب لا يفهم إلا 0 و 1
- نظام العد الثنائي أساس تخزين الأعداد
- أمثلة:

$$\begin{aligned}1_{10} &= 1_2 \\5_{10} &= 101_2 \\15_{10} &= 1111_2\end{aligned}$$

من الثنائي للعشري

- نحسب قيمة كل خانة:

$$1011_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 11_{10}$$

من العشري للثنائي

- نقسم العدد على 2 عدة مرات بشكل عودي ونأخذ قيم البواقي لناتج القسمة الصحيحة.
- مثال:

```
13 / 2 = 6 , remains = 1 (الخانه الأقل أهمية)
6 / 2 = 3, remains = 0
3 / 2 = 1, remains = 1
1 / 2 = 0, remains = 1 (الخانه الأكثر أهمية)

13 decimal = 1101 binary
```

Binary to Decimal

128	64	32	16	8	4	2	1
1	0	1	1	0	1	0	1
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

Decimal Value

181

Decimal to Binary



Remaining Value

0

Calculated Decimal Value

93

Two's Complement

-128	64	32	16	8	4	2	1
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Row A Unsigned

0

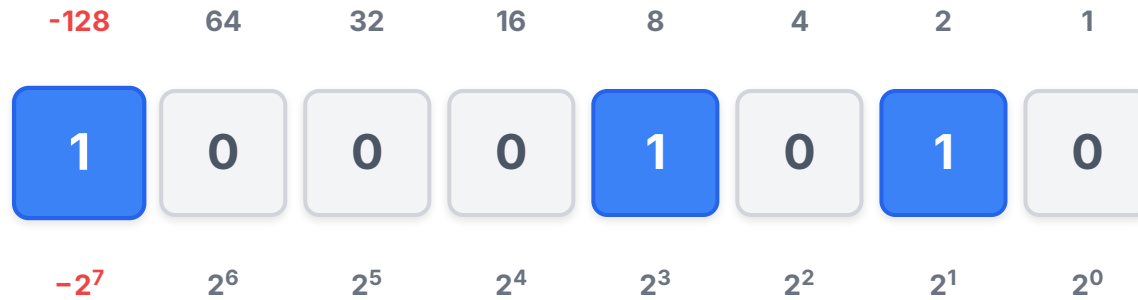
Signed: 0

Row B Unsigned

0

Signed: 0

الأعداد ذات الإشارة بطريقة المتمم الثاني Two's Complement



Red bit = Sign bit (Two's complement). ON → Negative number.

Unsigned

138

Signed (2's Complement)

-118

[illegible]

Exponent (8 bits)

Mantissa (23 bits)



عمليات الجمع والطرح بالثنائي

عمليات الجمع في النظام الثنائي تشبه الجمع في النظام العشري، لكن باستخدام رقمين فقط: 0 و 1. القاعدة الأساسية:

$$1 = 0 + 1$$

$$1 = 1 + 0$$

$$0 = 0 + 0$$

$$10 = 1 + 1 \leftarrow \text{هنا ينتج 0 ويُحْمَل 1 (Carry)}$$

•

قواعد الحمل (Carry) في الجمع الثنائي

• إذا كان الناتج 2 (أي $1 + 1$) نكتب 0 ونضيف 1 كحمل للخانة التالية.

• إذا كان $1 + 1 + 1$ (مع حمل سابق) يكون الناتج 11 نكتب 1 ويحمل 1.

أمثلة على الجمع الثنائي

(1) مثال بسيط

$$\begin{array}{r} 1 \\ 0110 \\ +0011 \\ \hline 1001 \end{array}$$

الخطوات:

- $1 = 1 + 0$
- $10 = 1 + 1 \rightarrow$ نكتب 0 ونحمل 1
- $10 = 0 + 1 + 1$ (الحمل) 1 $\rightarrow$ نكتب 0 ونحمل 1
- $1 = 0 + 0 + 1$ (الحمل) 1

مثال مع حمل مستمر

```
  111
 1011
+0111
-----
10010
```

الشرح:

- $10 = 1 + 1 \rightarrow$ نكتب 0 ونحمل 1
- $11 = 1 + 1 + 1$ (حمل) $\rightarrow$ نكتب 1 ونحمل 1
- $10 = 1 + 0 + 1$ (حمل) $\rightarrow$ نكتب 0 ونحمل 1
- $10 = 0 + 1 + 1$ (حمل) $\rightarrow$ نكتب 0 ونحمل 1
- نكتب الحمل الأخير في البداية $\rightarrow 1$

مفهوم الفيض و النقصان Overflow و Underflow



ما هو الفيض (Overflow)؟

ما هو الفيض؟

يحدث الفيض عندما ينتج عن العملية الحسابية الرقمية التي ينتج عنها رقم أكبر من الحد الأقصى لنوع البيانات.

مثال:

إضافة 1 إلى INT_MAX في نوع int.

مثال: فيض عند الجمع

```
int a = INT_MAX;  
int b = 1;  
int c = a + b; // فيض
```

النتيجة: تتجاوز قيمة الناتج الحد الأقصى وبذلك ننتقل للقيمة 1 (بشكل عام أصغر قيمة في حال كان العدد مؤشراً).

مثال على الفيض في الطرح

```
int a = INT_MAX;  
int b = -10;  
int c = a - b; // فيض
```

يشبه هذا الجمع لأن طرح عدد سالب يعادل إضافة عدد موجب أكبر مما يسمح به النوع.

النقصان (Underflow)؟

يحدث النقصان عندما ينتج عن العملية الحسابية رقم **أقل من الحد الأدنى** لنوع البيانات.

مثال:

طرح 1 من INT_MIN.



مثال: نقصان عند الطرح

```
int a = INT_MIN;  
int b = 1;  
int c = a - b; // نقصان
```

النتيجة: تتجاوز القيمة الحد الأدنى وننتقل لأكبر قيمة.